

COURSE CODE/TITLE:	CPE010	SECTION:	CPE21S2
DATE PERFORMED:	July 10, 2026	DATE SUBMITTED:	July 14, 2026
NAME:	Johrel Louie T. Silao	INSTRUCTOR:	Engr. Jimlord M. Quejado
ACTIVITY NO. <u>3.1</u> Hands-On Activity 1.1 Basic C++ Programming			
1. PROCEDURE OUTPUT			

- Create a function that will take two numbers and display the sum.

```
int main () {
    std::cout << "The sum of all numbers is : " << sumOfTwoNum(10,22) << std::endl;

    int x = 500;
    int y = 400;
```

Figure 1.1 Code snippet: Function of two numbers and display the sum.

In the figure above, a function initiating the two integer values and displaying the sum.

```
The sum of all numbers is : 32
```

Figure 1.2 Output snippet: Function of two numbers and display the sum.

This is the output which resulted from the execution of the program.

- Create a function that will return whether variable A is greater than variable B.

```
bool isGreater(int a, int b){
    return a > b;
}
```

Figure 2 Code snippet: Function that will return whether variable A is greater than variable B.

This snippet shows the implementation of the “isGreater” function. It takes two integer values and returns a boolean value that is based on whether variable is greater than variable B.

- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

```
bool displayLogicalOperations(bool c, bool d) {
    std::cout << std::boolalpha;
    std::cout << "Logical Operations Table " << std::endl;
    std::cout << "Inputs: A = " << c << ", B = " << d << std::endl;
    std::cout << "Not A      : " << (!c) << std::endl;
    std::cout << "Not B      : " << (!d) << std::endl;
    std::cout << "Logical AND : " << (c && d) << std::endl;
    std::cout << "Logical OR  : " << (c || d) << std::endl;
    std::cout << "Logical XOR : " << (c ^ d) << std::endl;
    std::cout << "Logical NAND: " << !(c && d) << std::endl;
    std::cout << "Logical NOR: " << !(c || d) << std::endl;
    std::cout << "Logical XNOR: " << !(c ^ d) << std::endl;
    return true;
}
```

Figure 3.1 Code snippet: Function that will take Boolean values and display the result.

In this snippet, it shows the implementation of “displayLogicalOperations” function. It takes two boolean values, proceeds to the console to display the boolean as text, and then it will output the logical operations table.

```
Logical Operations Table
Inputs: A = true, B = false
Not A      : false
Not B      : true
Logical AND : false
Logical OR  : true
Logical XOR : 1
Logical NAND: true
Logical NOR: false
Logical XNOR: false
Function execution status: true
```

Figure 3.2 Output snippet: Function that will take boolean values and display the result.

In this snippet, it shows the terminal output that is generated by the calling, which shows the expected output of the program.

Sections	Answer
Header File Declaration Section	#include <iostream>
Global Declaration Section	
Class Declaration and Method Definition Section	<pre> int sumOfTwoNum(int fNum, int sNum); bool isGreater(int a, int b); bool displayLogicalOperations(bool c, bool d); class Triangle { private: double totalAngle, angleA, angleB, angleC; public: Triangle(double A, double B, double C); void setAngles(double A, double B, double C); const bool validateTriangle(); </pre>
Main Function	<pre> int main () { std::cout << "The sum of all numbers is : " << sumOfTwoNum(10,22) << std::endl; int x = 500; int y = 400; if (isGreater(x, y)) { std::cout << "x is greater than y" << std::endl; } else { std::cout << "x is not greater than y" << std::endl; } bool success = displayLogicalOperations(true, false); std::cout << "Function execution status: " << std::boolalpha << success << std::endl; Triangle set1(40,50,110); if(set1.validateTriangle()){ std::cout << "The shape is a valid Triangle \n."; } </pre>

	<pre> else{ std::cout << "The shape is NOT a valid Triangle \n."; } return 0; } </pre>
Method Definition	<pre> int sumOfTwoNum(int fNum, int sNum){ int sum = fNum + sNum; return sum; } bool isGreater(int a, int b){ return a > b; } bool displayLogicalOperations(bool c, bool d) { std::cout << std::boolalpha; std::cout << "Logical Operations Table " << std::endl; std::cout << "Inputs: A = " << c << ", B = " << d << std::endl; std::cout << "Not A : " << (!c) << std::endl; std::cout << "Not B : " << (!d) << std::endl; std::cout << "Logical AND : " << (c && d) << std::endl; std::cout << "Logical OR : " << (c d) << std::endl; std::cout << "Logical XOR : " << (c ^ d) << std::endl; std::cout << "Logical NAND: " << !(c && d) << std::endl; std::cout << "Logical NOR: " << !(c d) << std::endl; std::cout << "Logical XNOR: " << !(c ^ d) << std::endl; return true; } </pre>

Table 1-1. C++ Structure Code for Answer

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
nf.lrm' '--pid=Microsoft-MIEngine-Pid-lf1libbhc.tue' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
The shape is a valid Triangle
PS C:\Users\TIPQC\Documents\CPE010_Silao\Seatwork 1>
```

In this terminal output, it shows that the implementation of a class triangle was implemented. It includes the necessary header files, triangle class and assigned private variables. A public method was also made if the user wishes to change the values of the angles of the triangles, then, it will proceed to the function where it will validate the values if it is a triangle.

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.
2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
3. Create a C++ program that has a function that will calculate the distance between two points.
4. Modify the code given in ILO B and add the following functions:
 - a. A function to compute for the area of a triangle
 - b. A function to compute for the perimeter of a triangle
 - c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
1  #include <iostream>
2  #include <cmath>
3  #include <string>
4
5  void numSwap();
6  double kelvinToFahrenheit(double kelvin);
7  void calculateDistance();
8  double distanceCalculator(double x1, double x2, double y1, double y2);
9
10 class Triangle {
11 private:
12     double sideA, sideB, sideC;
13     double angleA, angleB, angleC;
14     double totalAngle;
15 public:
16     Triangle(double sA, double sB, double sC,
17             double A, double B, double C);
18     void setTriangle(double sA, double sB, double sC,
19                     double A, double B, double C);
20     bool validateTriangle();
21     double getArea();
22     double getPerimeter();
23     std::string triangleType();
24 };
25
```

Figure 4.1 Code snippet: Different Problems using the C++ Programming Language

In this snippet, it shows the function declaration and the class triangle that will be used for the main structure of the program.

```

26 int main() {
27     double kelvin;
28     Triangle tri(5, 6, 7, 40, 50, 90);
29     std::cout << "\n===== ";
30     std::cout << "\nProblem 1: Swap two numbers in different variables." << std::endl;
31     numSwap();
32     std::cout << "\nProblem 2: Conversion of temperature in Kelvin to Fahrenheit.\n" << std::endl;
33     std::cout << "Enter temperature you want to convert in Kelvin: ";
34     std::cin >> kelvin;
35     std::cout << "\nTemperature converted to Fahrenheit: "
36     << kelvinToFahrenheit(kelvin) << " Fahrenheit" << std::endl;
37     std::cout << "\n===== ";
38     std::cout << "\nProblem 3: Calculating the distance between two points" << std::endl;
39     calculateDistance();
40     std::cout << "\nProblem 4: A function calculating the area of a triangle, perimeter of a triangle, the triangle is acute-angled and obtuse"
41
42     if (tri.validateTriangle()) {
43         std::cout << "The triangle is valid.\n";
44         std::cout << "Perimeter : " << tri.getPerimeter() << std::endl;
45         std::cout << "Area      : " << tri.getArea() << std::endl;
46         std::cout << "Type       : " << tri.triangleType() << std::endl;
47         std::cout << "===== ";
48     }
49     else {
50         std::cout << "The triangle is NOT valid.\n";
51     }
52
53     return 0;
54 }
55

```

Figure 4.2 Code snippet: Different Problems using the C++ Programming Language

In this snippet, it shows the main structure of the program where it will execute the function declaration and its definition.

```

56 void numSwap() {
57     int a, b, temp;
58     std::cout << "\nEnter any integer value for A and B" << std::endl;
59     std::cout << "A = ";
60     std::cin >> a;
61     std::cout << "B = ";
62     std::cin >> b;
63     std::cout << "\nThe values of A and B Before Swapping.\n";
64     std::cout << "A = " << a << std::endl;
65     std::cout << "B = " << b << std::endl;
66     temp = a;
67     a = b;
68     b = temp;
69     std::cout << "\nThe values of A and B after swapping." << std::endl;
70     std::cout << "A = " << a << std::endl;
71     std::cout << "B = " << b << std::endl;
72     std::cout << "\n===== ";
73 }
74
75 double kelvinToFahrenheit(double kelvin) {
76     return (kelvin - 273.15) * 9 / 5 + 32;
77 }
78

```

Figure 4.3 Code snippet: Different Problems using the C++ Programming Language

In this snippet, this is where the function's defined as problem 1 (Swapping two numbers in different values) and problem 2 (conversion of kelvin to fahrenheit). Problem 1 consists of only manipulating the variable's values and problem 2 is simply calculated using the fixed value of kelvin and converting the given value to gain Fahrenheit.

```

79 void calculateDistance() {
80     double x1, x2, y1, y2;
81     std::cout << "Enter x1: ";
82     std::cin >> x1;
83     std::cout << "Enter y1: ";
84     std::cin >> y1;
85     std::cout << "Enter x2: ";
86     std::cin >> x2;
87     std::cout << "Enter y2: ";
88     std::cin >> y2;
89     std::cout << "\nDistance Calculated:" << std::endl;
90     std::cout << "X1 : " << x1 << std::endl;
91     std::cout << "Y1 : " << y1 << std::endl;
92     std::cout << "X2 : " << x2 << std::endl;
93     std::cout << "Y2 : " << y2 << std::endl;
94     std::cout << "Distance = " << distanceCalculator(x1, x2, y1, y2) << std::endl;
95     std::cout << "=====";
96 }
97
98 double distanceCalculator(double x1, double x2, double y1, double y2) {
99     return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
100 }
101
102
103 Triangle::Triangle(double sA, double sB, double sC,
104     double A, double B, double C)
105 {
106     sideA = sA;
107     sideB = sB;
108     sideC = sC;
109     angleA = A;
110     angleB = B;
111     angleC = C;
112     totalAngle = A + B + C;
113 }
114

```

Figure 4.4 Code snippet: Different Problems using the C++ Programming Language

In this snippet, this is the function definition of problem 3 (calculate the distance between two points). This function is simply calculated using the “Distance Formula.”

```

115 void Triangle::setTriangle(double sA, double sB, double sC,
116 | | | | | | | | double A, double B, double C)
117 {
118     sideA = sA;
119     sideB = sB;
120     sideC = sC;
121     angleA = A;
122     angleB = B;
123     angleC = C;
124     totalAngle = A + B + C;
125 }
126
127 bool Triangle::validateTriangle()
128 {
129     return totalAngle == 180;
130 }
131
132 double Triangle::getPerimeter()
133 {
134     return sideA + sideB + sideC;
135 }
136
137 double Triangle::getArea()
138 {
139     double s = getPerimeter() / 2;
140     return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
141 }
142
143 std::string Triangle::triangleType()
144 {
145     if (angleA < 90 && angleB < 90 && angleC < 90)
146         return "Acute-Angled";
147     else if (angleA > 90 || angleB > 90 || angleC > 90)
148         return "Obtuse-Angled";
149     else
150         return "Others";
151 }

```

Figure 4.5 Code snippet: Different Problems using the C++ Programming Language

In this snippet, it shows the function definition of problem 4 (compute the area of a triangle, perimeter of a triangle, a triangle is acute-angled, obtuse-angled and 'others'). All sub problems were calculated by using their respective formulas in the triangle.


```

=====
Problem 1: Swap two numbers in different variables.

Enter any integer value for A and B
A = 23
B = 56

The values of A and B Before Swapping.
A = 23
B = 56

The values of A and B after swapping.
A = 56
B = 23

=====
Problem 2: Conversion of temperature in Kelvin to Fahrenheit.

Enter temperature you want to convert in Kelvin: 21

Temperature converted to Fahrenheit: -421.87 Fahrenheit

=====
Problem 3: Calculating the distance between two points
Enter x1: 34
Enter y1: 56
Enter x2: 89
Enter y2: 64

Distance Calculated:
X1 : 34
Y1 : 56
X2 : 89
Y2 : 64
Distance = 55.5788

=====
Problem 4: A function calculating the area of a triangle, perimeter of a triangle, the triangle is acute-angled and obtused-angled

The triangle is valid.
Perimeter : 18
Area      : 14.6969
Type      : Others
=====

```

Figure 4.5 Output snippet: Different Problems using the C++ Programming Language

This is the terminal output that correctly prints the expected output of the program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.